

■ Python Refresher

Complete Notes — Variables to Loops

#	Topic
1	Variables & Data Types
2	Operators
3	Strings & Slicing
4	String Methods
5	Lists
6	Tuples
7	Sets
8	Dictionaries
9	Loops (for / while)
10	Conditionals & Control Flow

1 · Variables

Naming Rules

- Must start with a letter or underscore (_)
- Cannot start with a digit
- Case-sensitive: myVar ≠ myvar
- Can contain letters, digits, and underscores

```
Python
1 x = 100          # single variable
2 x, y, z = 10, 20, 30 # multiple assignment
```

2 · Data Types

Type	Example	Description
str	'hello'	Text / string
int	42	Whole number
float	3.14	Decimal number
bool	True/False	Boolean value
None	None	Absence of value
complex	2+3j	Complex number

```
Python
1 """ or ''' # multi-line string
2 type(var)  # check type of a variable
```

3 · Type Casting

Implicit — Python converts automatically (e.g. int + float → float)

Explicit — you convert manually:

```
Python
1 int('42')    # 42
2 float(5)     # 5.0
3 str(100)     # '100'
4 bool(0)      # False
```

■ Any non-zero number / non-empty string is Truthy in Python.

4 · Operators

Arithmetic Operators

Op	Meaning	Example	Result
+	Addition	3 + 2	5
-	Subtraction	5 - 3	2
*	Multiplication	4 * 2	8
/	Division	7 / 2	3.5
//	Floor Division	7 // 2	3
%	Modulus	7 % 2	1
**	Exponent	2 ** 3	8

Comparison Operators

```

Operators
1 == != < <= > >=

```

■ All comparison operators return a boolean (True / False).

Assignment Operators

```

Operators
1 = += -= /= *= %=

```

Logical Operators

and	or
T T → T	T T → T
T F → F	T F → T
F T → F	F T → T
F F → F	F F → F

not — negates the boolean: not True → False

Membership & Identity Operators

Membership: in, not in — check if element exists in a sequence

Identity: is, is not — check if two variables point to the same object

```

Python
1 3 in [1, 2, 3] # True
2 x is None      # True if x is NoneType

```

5 · Strings

String Slicing [start : stop : step]

- **stop** index is NOT included
- Negative indices count from the end

```
Python
1 name = 'Rohi Bindal'
2 name[0:2] # 'Ro' - chars at index 0, 1
3 name[:5] # 'Rohi '
4 name[2:] # 'hi Bindal'
5 name[:] # full copy
6 name[-7:-1] # 'Bindal'
7 name[::-1] # reverse the string
```

■ Tip: String indices: 0 1 2 3 4 ... (left to right) Negative: -11 -10 ... -1 (right to left)

String Methods

Method	Description
.upper()	Convert all chars to uppercase
.lower()	Convert all chars to lowercase
.title()	Capitalise first letter of each word
.capitalize()	Capitalise only first character
.swapcase()	Swap upper ↔ lower case
.find(sub)	Return index of first occurrence (-1 if not found)
.replace(old, new)	Replace occurrences (1st by default)
.strip()	Remove leading & trailing whitespace
.lstrip()	Remove leading whitespace/chars
.rstrip()	Remove trailing whitespace/chars
.count(sub)	Count how many times sub appears
.isupper()	True if all chars are uppercase (boolean)
.islower()	True if all chars are lowercase (boolean)

■ `replace('Samson','Dhoni')` replaces 1st occurrence; pass a 3rd arg to limit how many.

6 · Lists

Property	Value
Ordered	✓ Yes
Mutable	✓ Yes — elements can be changed
Heterogeneous	✓ Yes — mixed types allowed
Duplicates	✓ Yes — allowed

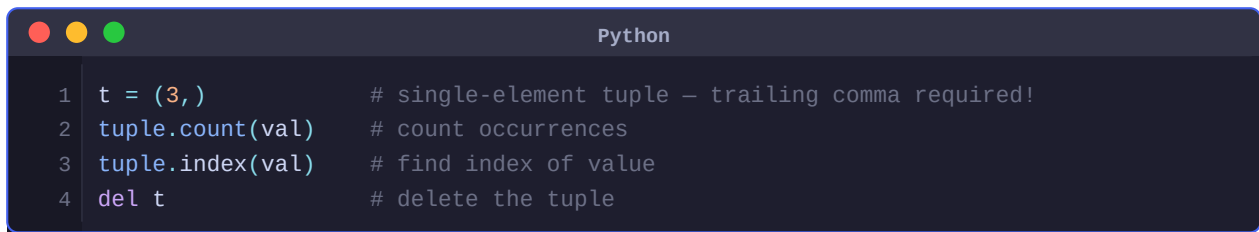
List Operations

```
Python
1 ipl_team = ['RCB', 'MI', 10, 20, True]
2
3 ipl_team[1]          # 'MI' — access by index
4 ipl_team[1][0]       # 'M' — nested index
5
6 ipl_team.append(20)   # add one element at end
7 ipl_team.extend(list2) # add all elements of list2
8 ipl_team.insert(3, 'Dhoni') # insert at index 3
9
10 len(ipl_team)        # length of list
11
12 ipl_team.remove(10)   # remove by value
13 ipl_team.pop(8)       # remove by index (returns item)
14 ipl_team.clear()      # remove all elements
15 del ipl_team          # delete the list entirely
16
17 ipl_team.copy()       # shallow copy
18 ipl_team.reverse()    # reverse in place
```

■ Tip: `copy()` is a shallow copy — nested objects are still shared. Use `copy.deepcopy()` for a fully independent clone.

7 · Tuples

Property	Value
Ordered	✓ Yes
Immutable	✓ Yes — cannot be changed after creation
Heterogeneous	✓ Yes
Duplicates	✓ Yes — allowed
Speed	■ Faster than lists
Memory	■ Uses less memory than lists

A screenshot of a Python IDE window titled "Python". The window has three colored window control buttons (red, yellow, green) in the top-left corner. The code is displayed on a dark background with light-colored text. Line numbers 1 through 4 are on the left. The code consists of four lines: a tuple assignment, a count operation, an index operation, and a deletion statement, each followed by a comment.

```
1 t = (3,)          # single-element tuple – trailing comma required!
2 tuple.count(val)   # count occurrences
3 tuple.index(val)   # find index of value
4 del t             # delete the tuple
```

■ *Without the trailing comma, Python treats (3) as just the integer 3 in parentheses.*

8 · Sets

Property	Value
Ordered	✗ No — order not guaranteed
Mutable	✓ Yes
Duplicates	✗ No — automatically removed
Heterogeneous	✓ Yes

```
Python
1 s = {1, 2, 3}
2
3 s.add(4)           # add single element
4 s.update([4, 5, 6]) # add multiple elements
5 s.remove(2)        # remove — raises error if not found
6 s.discard(2)       # remove — no error if missing
7
8 s.union(b)         # elements in s OR b
9 s.intersection(b)  # elements in s AND b
10 s.symmetric_difference(b) # in s OR b but NOT both
```

9 · Dictionary

Property	Value
Mutable	✓ Yes
Ordered	✓ Insertion order kept (Python 3.7+)
Heterogeneous	✓ Keys & values can be any type
Unique Keys	✓ Keys must be unique; values can repeat

```
Python
1 student = {'name': 'Rohi', 'age': 25}
2
3 student.get('name') # 'Rohi' — safe (no KeyError)
4 student['age'] = 25  # access or update by key
5 student['height'] = 5.6 # add new key
6
7 student.keys()       # dict_keys(['name', 'age', 'height'])
8 student.values()     # dict_values(['Rohi', 25, 5.6])
9 student.items()      # dict_items([('name', 'Rohi'), ...])
```

■ Use `.get()` over `[]` when a key might not exist — returns `None` instead of `KeyError`.

Built-in Data Types — Quick Comparison

Type	Ordered	Mutable	Duplicates	Syntax
------	---------	---------	------------	--------

List	Yes	Yes	Yes	[]
Tuple	Yes	No	Yes	()
Set	No	Yes	No	{ }
Dictionary	Yes*	Yes	Keys: No	{ k:v }
String	Yes	No	Yes	' ' or " "

* Dicts maintain insertion order since Python 3.7

10 · Loops

for Loop

```
Python
1 # Basic range loop
2 for i in range(10):
3     print(i)          # prints 0 to 9
4
5 # Print even numbers from a list
6 nums = [1, 2, 3, 4, 5, 6, 7, 8]
7 for i in nums:
8     if i % 2 == 0:
9         print(i)
10
11 # Iterate over a dictionary
12 for key in data.keys():    print(key)
13 for value in data.values(): print(value)
14 for key, value in data.items(): print(key, value)
```

■ Tip: `range(start, stop, step)` — `stop` is NOT included. `range(1,6)` → 1,2,3,4,5 | `range(0,10,2)` → 0,2,4,6,8

while Loop

```
Python
1 i = 1
2 while i < 10:
3     print(i)
4     i += 1    # always update the variable to avoid infinite loop!
```

■ A while loop without an update will run forever. Always ensure the condition eventually becomes False.

print() — end parameter

```
Python
1 print('*', end='')    # no newline – stays on same line
2
3 # Nested loop – triangle pattern
4 for i in range(1, 6):
5     for j in range(i):
6         print('*', end='')
7     print()           # newline after each row
```

11 · Conditionals

```
Python
1 if condition:
2     # executes when condition is True
3 elif another_condition:
4     # executes when elif is True
5 else:
6     # executes when none of the above match
```

12 · Loop Control Statements

```
Python
1 break      # exit the loop immediately
2 continue   # skip current iteration, continue loop
```

Statement	Behavior
break	Jumps out of the loop entirely
continue	Skips current iteration, continues loop
pass	No-op placeholder — does nothing

Bonus · Common Python Patterns

```
Python
1 # List comprehension — compact for-loop
2 evens = [i for i in range(10) if i % 2 == 0]
3
4 # Ternary / inline if
5 result = 'even' if x % 2 == 0 else 'odd'
6
7 # Enumerate — loop with index
8 for idx, val in enumerate(my_list):
9     print(idx, val)
10
11 # Zip — loop two lists together
12 for a, b in zip(list1, list2):
13     print(a, b)
14
15 # Check type (preferred)
16 isinstance(x, int)  # better than type(x) == int
```

■ Tip: Master list comprehensions early — they replace most simple for-loops in real Python code.